PART I

- This PA3 is based on PA1.
- 在"Micrium/Software/uCOS-II/Source/ucos_ii.h"的 task_para_set 中新增參數用於儲存 Block、Preemption、資源 Lock、Unlock 的時間。

```
82  typedef struct task_para_set {
83      INT16U TaskID;
84      INT16U TaskArriveTime;
85      INT16U TaskExecutionTime;
86      INT16U TaskPeriodic;
87      INT16U TaskNumber;
88      INT16U TaskPriority;
89      INT16U TaskCount;
90      INT16U TaskRemainTime;
91      INT16U TaskDeadLine;
92      INT16U TaskResponseTime;
93      INT16U TaskDelay;
94      INT16U TaskStartTime;
95      INT16U TaskReadyTime;
96      INT16U TaskBlockTime;
97      INT16U TaskPreemptiveTime;
98      INT16U R1Lock;
99      INT16U R1UnLock;
100     INT16U R2Lock;
101     INT16U R2UnLock;
102 } task_para_set;
```

- 在"Microsoft/Windows/Kernel/OS2/app_hooks.c"的 InputFile()中初始化上述參數。

```
95      while (!feof(fp)) {
96          i = 0;
97          memset(str, 0, sizeof(str));
98          fgets(str, sizeof(str) - 1, fp);
99          ptr = strtok_s(str, " ", &pTmp);
100
101         while (ptr != NULL) {
102             TaskInfo[i] = atoi(ptr);
103             ptr = strtok_s(NULL, " ", &pTmp);
104             if (i == 0) {
105                 TASK_NUMBER++;
106                 TaskParameter[j].TaskID = TASK_NUMBER;
107             }
108             else if (i == 1) {
109                 TaskParameter[j].TaskArriveTime = TaskInfo[i];
110                 TaskParameter[j].TaskStartTime = TaskInfo[i];
111             }
112             else if (i == 2) {
113                 TaskParameter[j].TaskExecutionTime = TaskInfo[i];
114                 TaskParameter[j].TaskRemainTime = TaskInfo[i];
115             }
116             else if (i == 3) TaskParameter[j].TaskPeriodic = TaskInfo[i];
117             else if (i == 4) TaskParameter[j].R1Lock = TaskInfo[i];
118             else if (i == 5) TaskParameter[j].R1UnLock = TaskInfo[i];
119             else if (i == 6) TaskParameter[j].R2Lock = TaskInfo[i];
120             else if (i == 7) TaskParameter[j].R2UnLock = TaskInfo[i];
121
122             TaskParameter[j].TaskDeadLine = TaskParameter[j].TaskArriveTime + TaskParameter[j].TaskPeriodic;
123             TaskParameter[j].TaskReadyTime = 0;
124             TaskParameter[j].TaskBlockTime = 0;
125             TaskParameter[j].TaskPreemptiveTime = 0;
```

- 在"Microsoft/Windows/Kernel/OS2/app_hooks.c"的 App_TimeTickHook ()中每次 Tick 時檢查所有任務，只要任務的週期到了，就設成 Ready 並重置 TaskRemainTime 時間。

```
354     #if OS_TIME_TICK_HOOK_EN > 0
355     void  App_TimeTickHook (void)
356     {
357     #if (APP_CFG_PROBE_OS_PLUGIN_EN == DEF_ENABLED) && (OS_PROBE_HOOKS_EN > 0)
358         OSProbe_TickHook();
359     #endif
360         OS_TCB* ptcb = OSTCBList;
361         int timeTag = OSTimeGet() + 1;
362
363         while (ptcb) {
364             task_para_set* task_data = (task_para_set*)ptcb->OSTCBExtPtr;
365                 if (task_data && timeTag >= task_data->TaskStartTime) {
366                     int gap = timeTag - task_data->TaskStartTime;
367                     if (task_data->TaskPeriodic > 0 && (gap % task_data->TaskPeriodic) == 0) {
368                         task_data->TaskRemainTime = task_data->TaskExecutionTime;
369                         task_data->TaskReadyTime = 1;
370                     }
371                 }
372             ptcb = ptcb->OSTCBNext;
373         }
374     }
375     #endif
376     #endif
```

- 在" Micrium/Software/uCOS-II/Source/os_core.c"
  - OSIntExit():將後面列印改為 Block time、Preemption time，並在顯示完重置任務參數。

```
756                     if (OSTCBHighRdy->OSTCBPrio != 63) {
757                         LOG_print(3, "./Output.txt", "%2d  Completion\ttask(%2d)(%2d)\ttask(%2d)(%2d)\t%d\t%d\t%d\t\n",
758                             OSTimeGet(),
759                             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskID,
760                             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskCount,
761                             ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))->TaskID,
762                             ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))->TaskCount,
763                             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskResponseTime - 1,
764                             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskBlockTime,
765                             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskPreemptiveTime
766                         );
767                     }
768                     else {
769                         LOG_print(3, "./Output.txt", "%2d  Completion\ttask(%2d)(%2d)\ttask(%2d)\t%d\t%d\t%d\n", OSTimeGet(),
770                             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskID,
771                             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskCount,
772                             63,
773                             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskResponseTime - 1,
774                             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskBlockTime,
775                             ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskPreemptiveTime
776                         );
777                     }
778
779                     ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskBlockTime = 0;
780                     ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskPreemptiveTime = 0;
781                     ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskReadyTime = 0;
782                     ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskCount++;
783
784                     OSTCBCur->CompletedFlag = 0;
785                     ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskArriveTime -= ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskPeriodic;
786                 }
```

✧ OSTimeTick ():檢查任務目前是否有資格被排入執行，但因優先權原因沒被執行，並記錄是被 Block 還是 Preemption。

```
if (ptcb->OSTCBStat == OS_STAT_RDY &&
    ptcb != OSTCBCur &&
    task_data->TaskRemainTime > 0 &&
    currentTime > task_data->TaskStartTime &&
    task_data->TaskReadyTime == 1 &&
    currentTime > task_data->TaskArriveTime &&
    OSPrioCur != 63) {

    if (ptcb->OSTCBPrio < OSTCBCur->OSTCBPrio) task_data->TaskBlockTime++;
    else task_data->TaskPreemptiveTime++;
}
```

✧ OS_Sched ():新增 Preemption， OSTCBCur 還沒結束也沒錯過 deadline 而排程器選出更高優先度任務（OSTCBHighRdy）要跑；將後面列印改為 Block time、Preemption time，並在顯示完重置任務參數。

```
if (OSTCBCur->CompletedFlag != 1 && missDeadline == 0 && ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr)) != NULL) {
    LOG_print(3, "./Output.txt", "%2d  Preemption\ttask(%2d)(%2d)\ttask(%2d)(%2d)\n",
        OSTimeGet(),
        ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskID,
        ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskCount,
        ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))->TaskID,
        ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))->TaskCount
    );
}
if (OSTCBCur->CompletedFlag == 1 && missDeadline == 0) {
    if (OSTCBHighRdy->OSTCBPrio != 63) {
        LOG_print(3, "./Output.txt","%2d  Completion\ttask(%2d)(%2d)\ttask(%2d)(%2d)\t%d\t%d\t%d\t\n",
            OSTimeGet(),
            ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskID,
            ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskCount,
            ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))->TaskID,
            ((task_para_set*)(OSTCBHighRdy->OSTCBExtPtr))->TaskCount,
            ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskResponseTime,
            ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskBlockTime,
            ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskPreemptiveTime
        );
    }
    else {
        LOG_print(3, "./Output.txt","%2d  Completion\ttask(%2d)(%2d)\ttask(%2d)\t%d\t%d\t%d\n",
            OSTimeGet(),
            ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskID,
            ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskCount,
            63,
            ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskResponseTime,
            ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskBlockTime,
            ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskPreemptiveTime
        );
    }

    ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskBlockTime = 0;
    ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskPreemptiveTime = 0;
    ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskReadyTime = 0;
    ((task_para_set*)(OSTCBCur->OSTCBExtPtr))->TaskCount++;
    OSTCBCur->CompletedFlag = 0;
}
```

✧ OS_SchedNew ():初始化所有任務的延遲到 TaskStartTime，確保每個任務在指定時間後才會開始執行。

```
static  void  OS_SchedNew(void)
{
#if OS_LOWEST_PRIO <= 63u                   /* See if we support up to 64 tasks                   */
    INT8U   y;

    static INT8U check = 0u;

    if (check == 0u) {
        check = 1u;

        OS_TCB* ptcb;
        ptcb = OSTCBList;

        while (ptcb != NULL && ptcb->OSTCBExtPtr != NULL) {
            if (((task_para_set*)(ptcb->OSTCBExtPtr))->TaskStartTime > 0) {
                ptcb->OSTCBDly = ((task_para_set*)(ptcb->OSTCBExtPtr))->TaskStartTime;
                if ((OSRdyTbl[ptcb->OSTCBY] &= ~ptcb->OSTCBBitX) == 0u) OSRdyGrp &= ~ptcb->OSTCBBitY;
            }
            ptcb = ptcb->OSTCBNext;
        }
    }


    y = OSUnMapTbl[OSRdyGrp];
    OSPrioHighRdy = (INT8U)((y << 3u) + OSUnMapTbl[OSRdyTbl[y]]);
```

- 在" Microsoft/Windows/Kernel/OS2/main.c"的 task ()中模擬資源 Lock & Unlock。
  - ✧ 如果 R1 有設定、且現在時間到達 R1Lock -> 鎖 R1
    - ✓ 當 exeTime == task_data->R1Lock 時，代表要鎖住 R1。呼叫 OSSchedLock() 來鎖資源。
  - ✧ 如果 R2 有設定、且現在時間到達 R2Lock -> 鎖 R2
    - ✓ 邏輯同上，只是換成 R2。到 R2Lock 時呼叫 OSSchedLock() 把排程與資源鎖住。
  - ✧ 如果 R1 到達解鎖時間 -> 解鎖 R1
    - ✓ 當 exeTime == task_data->R1UnLock 時，代表時間到達 R1UnLock，釋放 R1，呼叫 OSSchedUnlock() 讓排程恢復正常。
  - ✧ 如果 R2 到達解鎖時間 -> 解鎖 R2
    - ✓ 邏輯同上，時間到達 R2UnLock 時釋放 R2，呼叫 OSSchedUnlock() 讓排程恢復正常。

```
101         while (missDeadline == 0) {
102             task_data->TaskReadyTime = 1;
103             while (task_data->TaskRemainTime > 0 && missDeadline == 0) {
104                 if (missDeadline == 1) break;
105 
106                 exeTime = task_data->TaskExecutionTime - task_data->TaskRemainTime;
107 
108 #if PROTOCOL == NPCS
109                 if (task_data->R1UnLock != 0 && exeTime == task_data->R1Lock) {
110                     LOG_print(3, "./Output.txt", "%2d  LockResource\ttask(%2d)(%2d)\tR1\n",
111                         OSTimeGet(),
112                         task_data->TaskID,
113                         task_data->TaskCount
114                     );
115                     OSSchedLock();
116                 }
117                 else if (task_data->R2UnLock != 0 && exeTime == task_data->R2Lock) {
118                     LOG_print(3, "./Output.txt", "%2d  LockResource\ttask(%2d)(%2d)\tR2\n",
119                         OSTimeGet(),
120                         task_data->TaskID,
121                         task_data->TaskCount
122                     );
123                     OSSchedLock();
124                 }
125 
126                 if (task_data->R1UnLock != 0 && exeTime == task_data->R1UnLock) {
127                     LOG_print(3, "./Output.txt", "%2d  UnlockResource\ttask(%2d)(%2d)\tR1\n",
128                         OSTimeGet(),
129                         task_data->TaskID,
130                         task_data->TaskCount
131                     );
132                     OSSchedUnlock();
133                 }
134                 else if (task_data->R2UnLock != 0 && exeTime == task_data->R2UnLock) {
135                     LOG_print(3, "./Output.txt", "%2d  UnlockResource\ttask(%2d)(%2d)\tR2\n",
136                         OSTimeGet(),
137                         task_data->TaskID,
138                         task_data->TaskCount
139                     );
140                     OSSchedUnlock();
141                 }
142 
143 #endif
144                 LOG_print(3, "./Output.txt", "%2d  task(%2d) is running\t\n", OSTimeGet(), task_data->TaskID);
145                 timeTag = OSTimeGet();
```

PART II

Not Finished.

PART III

NPCS (Non-Preemptive Critical Section)

- How it works:
  - 任務在進入臨界區時呼叫 OSSchedLock()，排程器被鎖住，系統不會切換到其他任務。
  - 任務要離開時呼叫 OSSchedUnlock()，排程器恢復運作。
  - 臨界區期間整個 CPU 都被這個任務占用。
- How it avoids deadlock:
  - 在 NPCS 下，只要一個任務鎖住排程器，禁止任務切換，其他任務根本不能執行，所以不會形成循環等待，也不會死結。
- Advantages:
  - 運作簡單、容易實作。
  - 不需要複雜的優先權計算。
  - 不會有死結。
- Disadvantages:
  - 臨界區期間所有任務被迫等待。
  - 無法處理即時系統的高優先權搶佔。
  - 臨界區長度大會造成系統延遲上升。

CPP (Ceiling Priority Protocol)

- How it works:
  - 每個資源都會被設定一個 Priority Ceiling，所有會使用這個資源的任務的最高的優先權。
  - 當任務鎖資源時，它的優先權被臨時提高到 ceiling level，其他優先權低於 ceiling level 的任務不能搶奪或進入其他會造成衝突的資源，任務離開臨界區後，優先權恢復正常。
- How it avoids deadlock:
  - 任務持有資源期間，優先權被提升，因此不會被其他任務搶佔，如果任務的即將獲得某資源會造成死結，PCP 直接阻止它進入。
- Advantages:
  - 可以避免死結。
  - 避免優先權反轉。
  - 即時性較 NPCS 好。
  - 支援多資源情況下的安全同步。

- Disadvantages:
  - 計算天花板優先權需要分析所有任務。
  - 任務進入臨界區時的優先權提升需要額外 overhead。
  - 相比 NPCS 較難 debug。